

C++ Programming

Trainer : Rajiv K

Email: rajiv.kamune@sunbeaminfo.com



Agenda

- static data member
- static member function
- constant data member
- constant (data member, member function, object)
- mutable keyword
- modular Approach
- Composition
- Template



static data member

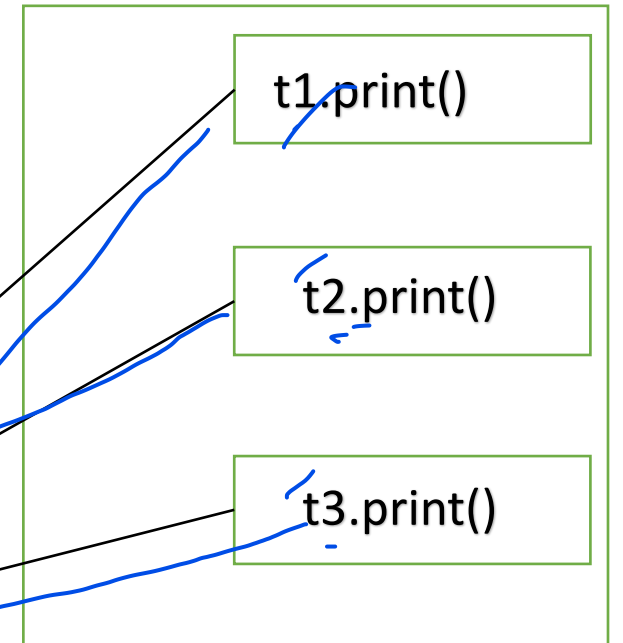
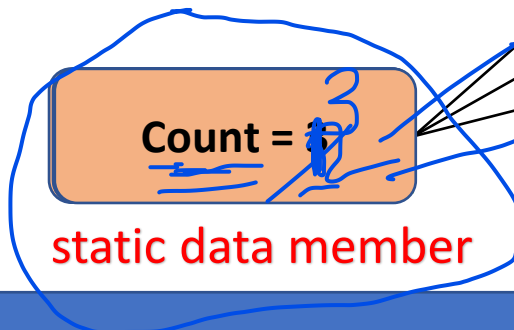
- Only **one copy** of that **member** is **created** for the **entire class** and is shared by **all the objects** of that class, no matter how many objects are created.
- It is initialized before any object of this class is being created, even before main() starts.
- It is **visible only within the class**, but its **lifetime is the entire program**
- **Syntax:**
 - static data-type data-member-name;
- **Example:**
 - static int num1;
 - *static members are **only declared in a class declaration, not defined.***
 - *They must be **explicitly defined outside the class** using the scope resolution operator.*
 - Initialized static and global variable get space on **Data segment.**



Static data member

```
1. class Test
2. {   static int count;
3.     public:
4.         void print()
5.         {
6.             cout<<"Count:"<<++count;
7.         }
8. };
9. int Test::count=0;    initialization or definition
10. int main()
11. {
12.     Test t1,t2,t3;
13.     cout<<"count:"<<Test::count<<endl;
14.     t1.print();
15.     t2.print();
16.     t3.print();
17.     // cout<<"Count "<<Test::count;
18.     return 0;
19. }
```

$0+1 \Rightarrow 1+1 \Rightarrow 2+1 = 3$



static member function of class

- Just like the static data members or static variables inside the class, static member functions also **does not depend on object of class.**
- Static **member functions** are **allowed** to **access** only the **static data members** or other **static member functions.**
- We are allowed to invoke a static member function using the object and the '.' operator but it is **recommended to invoke** the static members using the class name and the **scope resolution operator(::).**

- **Syntax:**

- static return-type member-function-name
- { //Function Body};

- **Example:**

- static void print()
- {
- cout<<"Inside print function";
- }



static member function of class

- Static member function do not get this pointer.
- You can not declare static member function as virtual.
- You can not declare the static member function as const.

- `class Test`

- `{ static int count;`
- `public:`
- `static void print()`
- `{ cout<<"count: "<<++Test::count;}`
- `};`

- `int Test::count=0;`

- `int main()`

- `{`
- `Test::print();`
- `return 0;`
- `};`



constant data member

```
const int num1=10;
```

```
num1=20; //not ok
```

- If we **do not want to modify value** of the **variable** then const keyword is used.
- constant variable is also called as **read only variable**.
- The value of such variable should be known **at compile time**.
- In C++ , **Initializing constant variable is mandatory**
- To initialize the const data member of a class we need to use **the member initializer list**.
 - `const int num=10; //allowed in C++`
 - `const int num; // not allowed in C++`



constant member function of class

- A constant member function of a class that never changes any class data members, and it also does not call any non-const function.
- It is also known as the **read-only function**.
- We can create a **constant member function of a class by adding the const keyword after the name of the member function**.
- We can not declare following function constant:
 1. Global Function
 2. Static Member Function
 3. Constructor
 4. Destructor
- **Syntax**
 1. `return_type fuction_name () const //constant function declaration`
 2. `{`
`//Function Body`
 3. `}`



constant object

- **objects** that need to be **unmodifiable** can be defined as constant objects.
- Member function that operates on **const objects**, **without modifying them**, must be **declared** with the **const** key word.
- The const key word must be specified after the parameter list in function prototype and function definition.
- The **constructor and destructor** of const object **do not need the const** key word.
- When a class contain the const data member it can't be initialized inside the body of constructor, because a const member by definition can not be assigned a new value and C++ dose not allow the initialization in the class definition.
- **Syntax:**
 - `const class-name object-name;`



mutable keyword

- **mutable keyword:** mutable is storage **class specifier**
- In case of constant member function you can't change the value of data member through it , it is not allowed in C++.
- Hence mutable keyword comes in picture, **mutable key word allows** to modify the state of **data member through constant member function**.
- **Syntax:**
- mutable data_type variable_name;
- **Example:**
- mutable int num=10;



Modular Programming

- If a function or group of function belonging together are put in separate module/source file, there **modular programming** is realized.
- Modules can be **compiled and tested separately** and they can be **imported in to other projects as well**.
- By using modular programming we can **achieve code reusability**.
- If we include header file in **angular bracket** (e.g `#include<filename.h>`) then **preprocessor try to locate and load header file from standard directory** only(/usr/include).
- If we include header file in **double quotes** (e.g `#include"filename.h"`) then preprocessor try to locate and load header file first **from current project directory** if not found then it try to locate and load from standard directory.



Modular Programming

- **Advantages Of Modular Programming:**

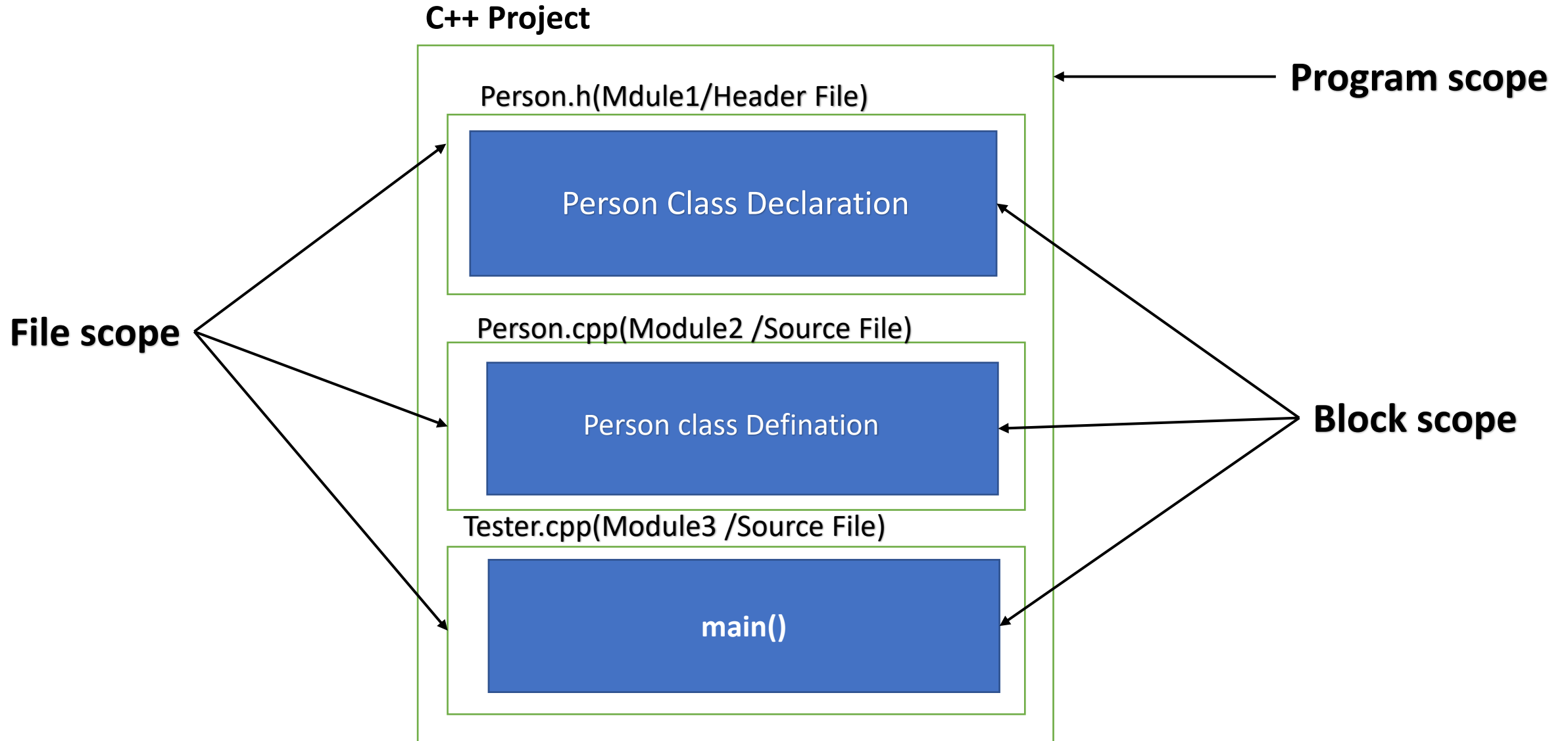
- Ease of Use
- Reusability
- Ease of maintenance

- **Header Guard**

```
#ifndef HEADER_FILE_NAME_H_  
#define HEADER_FILE_NAME_H_  
    //TODO : Type declaration here(Class declaration)  
#endif
```



Modular Programming Block Diagram



Advantages Of Modular Programming:

- **Ease of Use:** This approach allow simplicity, as rather than focusing on the entire thousand and millions of lines code in one go we can access it in the form of module. This allows ease in debugging the code and prone to less error.
- **Reusability:** It allows the user to reuse the functionality with different interface without typing the whole program again.
- **Ease of maintenance:** It helps in less collision at the time of working on module. Helps the team to work with proper collaboration while working on large application.



Association

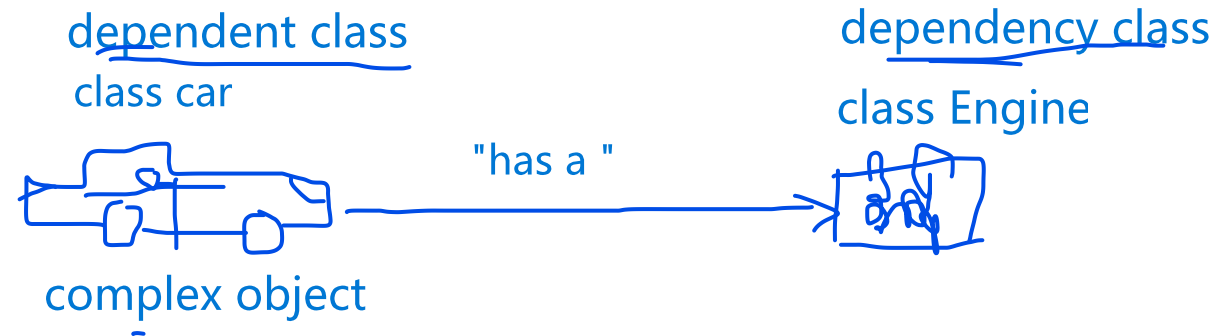
- If there is **has-a** relation between the classes, then we should use association.
- If object is a component of another object then it is called as association.

- **Example:**

- Person has-a date-of-birth
- Mobile has-a screen
- Car has-a engine

- Type of Association

- 1. Aggregation
- 2. Composition



Aggregation

- It is a special form of association whole-part relationship
- If dependency object can exist without dependent object, then it is called as aggregation.
- Consider the following example:
 - Department has-a faculty.
 - Dependent Object: Department
 - Dependency Object: Faculty



Composition

- It is also special form of association
- If dependency object dose not exist without dependent object then it is called as composition

- **Example:**

- Human has-a heart
- Dependent object: Human
- Dependency object: Heart



Template

- While calling the function we **pass the data type** too, so that **repetitive code writing can be avoided to reduce the programmers effort**.
- That means we are trying to write the **generic program** with the help of templates.
- **Example:**
- Write a function to **add two int num** and write a code to **add two double num** for this we need to write the **two separate functions** and we may call it as **function overloading** if they are having **same name but different signatures as follows**.

```
Case1:int sum (int num1,int num 2)           //Function to add 2 int nums  
        {return num1+num2;} //business logic
```

```
Case2:double sum(double num1,double num2) //Function to add 2 double nums  
        {return num1+num2;}//business logic
```



template

- In above code we can see we have written the repetitive code so to avoid this we use templates.
- Template gets resolve a compile time.
- template and typename or class are the key word we use to implement the templates.
- Type of templates
 - 1. function template
 - 2. class template
- **Syntax:**
- **template<typename T>**
- **Template<class T>**



Template

- **Example: function template**

```
1.  template<typename T>
2.  T sum(T num1, T num2)
3.  {return num1+num2 }
```

```
4.  int main()
```

```
5.  {
```

```
6.      int result1;
```

```
7.      double result2;
```

```
8.      result1=sum<int> (10,20) ;
```

```
9.
10.     result2=sum<double> (34.7,56.4) ;
```

```
11. }
```

//compiler will resolve this call
//as follows
int sum(int num1,int num2)
{return num1+num2}

//compiler will resolve this call //as
follows
double sum(double num1, double num2)
{return num1+num2}



Thank You

